SQL Documentation for Beginners

All examples use a simple `Employees` table with columns like `EmployeeID`, `Name`, `Department`, `Salary`, `JoiningDate`, `ManagerID`.

1. CREATE DATABASE

Syntax:

```
CREATE DATABASE database_name;
```

Example:

```
CREATE DATABASE CompanyDB;
```

Explanation:
Creates a new, empty database to store tables and other objects.
Purpose: To set up a dedicated storage space for a project or application before creating tables.
Real-time example: An HR system creates a `CompanyDB` database to hold all employee, payroll, and department data.
Tester's POV: Testers create a separate test database (e.g., `CompanyDB_QA`) so test data never touches production.

2. CREATE TABLE

Syntax:

```
CREATE TABLE table_name (
    column_name datatype,
    column_name datatype
);
```

Example:

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

Explanation:
Creates a new table with defined columns and their data types.
Purpose: To define the structure that will hold organized, structured records.
Real-time example: An onboarding module creates an `Employees` table to store details of every

new hire.
Tester's POV: Testers set up mock tables matching production schema to validate API/database automation scripts before real data is involved.

3. INSERT

Syntax:

```
INSERT INTO table_name (column_name, column_name)
VALUES (value1, value2);
```

Example:

```
INSERT INTO Employees (EmployeeID, Name, Department, Salary)
VALUES (1, 'Anitha', 'IT', 55000);
```

Explanation:
Adds a new row of data into a table.
Purpose: To record new information as it becomes available.
Real-time example: HR inserts a new row when Anitha joins the IT department.
Tester's POV: Testers insert seed/test data before running automation suites to ensure predictable, repeatable test conditions.

4. SELECT

Syntax:

```
SELECT column_name FROM table_name;
```

Example:

```
SELECT Name, Salary FROM Employees;
```

Explanation:
Retrieves specific columns of data from a table.
Purpose: To view and extract only the data you need for reporting or analysis.
Real-time example: A manager pulls the name and salary list to review the team's pay structure.
Tester's POV: Testers use `SELECT` to verify backend data matches what the UI or API response shows — a core step in data validation testing.

5. DISTINCT

Syntax:

```
SELECT DISTINCT column_name FROM table_name;
```

Example:

```sql
SELECT DISTINCT Department FROM Employees;
```

Explanation:
Returns only unique values, removing duplicates from the result.
Purpose: To quickly see the range of unique categories without repetition.
Real-time example: HR checks all distinct departments that currently exist in the company.
Tester's POV: Testers use `DISTINCT` to check for unwanted duplicate records created by faulty insert/update logic.

6. WHERE

Syntax:

```sql
SELECT column_name FROM table_name WHERE condition;
```

Example:

```sql
SELECT Name FROM Employees WHERE Department = 'IT';
```

Explanation:
Filters rows so only those matching the condition are returned.
Purpose: To narrow down large datasets to only the relevant records.
Real-time example: A recruiter lists only employees who belong to the IT department.
Tester's POV: Testers use `WHERE` to fetch exact test records by ID/email before validating them against expected results.

7. AND / OR / NOT

Syntax:

```sql
SELECT column_name FROM table_name WHERE condition1 AND condition2;
SELECT column_name FROM table_name WHERE condition1 OR condition2;
SELECT column_name FROM table_name WHERE NOT condition;
```

Example:

```sql
SELECT Name FROM Employees WHERE Department = 'IT' AND Salary > 50000;
```

Explanation:
Combines multiple conditions — `AND` needs both true, `OR` needs either true, `NOT` reverses a condition.
Purpose: To build precise, multi-condition filters for complex business rules.
Real-time example: Finance lists IT employees earning above 50,000 for a bonus review.
Tester's POV: Testers combine conditions to design boundary and negative test cases, like verifying records that should NOT appear in a filtered report.

8. BETWEEN

Syntax:

```
SELECT column_name FROM table_name WHERE column_name BETWEEN value1 AND value2;
```

Example:

```
SELECT Name FROM Employees WHERE Salary BETWEEN 40000 AND 60000;
```

Explanation:
Selects values that fall within a given range, including the endpoints.
Purpose: To filter data within a specific numeric or date range easily.
Real-time example: Payroll identifies employees in the mid-level salary band for a policy update.
Tester's POV: Testers use `BETWEEN` to validate boundary values (min/max limits) — a classic boundary value analysis technique.

9. IN

Syntax:

```
SELECT column_name FROM table_name WHERE column_name IN (value1, value2);
```

Example:

```
SELECT Name FROM Employees WHERE Department IN ('IT', 'HR');
```

Explanation:
Checks if a value matches any value in a given list.
Purpose: To match against multiple possible values without writing several OR conditions.
Real-time example: An event coordinator invites employees from either the IT or HR departments.
Tester's POV: Testers use `IN` to quickly validate that only expected status values (e.g., 'Active', 'Pending') exist in a dataset after a bulk operation.

10. LIKE

Syntax:

```
SELECT column_name FROM table_name WHERE column_name LIKE 'pattern';
```

Example:

```
SELECT Name FROM Employees WHERE Name LIKE 'A%';
```

Explanation:
Searches for a specified pattern in text using wildcards (`%` for many characters, `_` for one).
Purpose: To search for partial matches when the exact value is unknown.
Real-time example: A search bar finds all employees whose names start with "A".
Tester's POV: Testers use `LIKE` to verify search/filter features in the application actually query the database correctly.

11. IS NULL / IS NOT NULL

Syntax:

```
SELECT column_name FROM table_name WHERE column_name IS NULL;
SELECT column_name FROM table_name WHERE column_name IS NOT NULL;
```

Example:

```
SELECT Name FROM Employees WHERE ManagerID IS NULL;
```

Explanation:
Checks whether a column has no value (`NULL`) or has any value.
Purpose: To find missing or incomplete data in records.
Real-time example: HR finds employees with no assigned manager to fix reporting gaps.
Tester's POV: Testers check for `NULL` values to catch data integrity bugs, like mandatory fields left empty after form submission.

12. ORDER BY

Syntax:

```
SELECT column_name FROM table_name ORDER BY column_name ASC|DESC;
```

Example:

```
SELECT Name, Salary FROM Employees ORDER BY Salary DESC;
```

Explanation:
Sorts the result set in ascending (`ASC`, default) or descending (`DESC`) order.
Purpose: To present data in a meaningful, ranked order.
Real-time example: Management views the highest-paid employees first for a salary audit.
Tester's POV: Testers use `ORDER BY` to validate that UI sorting/pagination features match the correct database order.

13. GROUP BY

Syntax:

```sql
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Example:

```sql
SELECT Department, COUNT(*) FROM Employees GROUP BY Department;
```

Explanation:
Groups rows sharing the same value so aggregate functions can be applied per group.
Purpose: To summarize data by category instead of row by row.
Real-time example: HR counts how many employees work in each department.
Tester's POV: Testers use `GROUP BY` to cross-check dashboard/report counts (e.g., total records per category) against the actual database.

14. HAVING

Syntax:

```sql
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example:

```sql
SELECT Department, COUNT(*) FROM Employees GROUP BY Department HAVING COUNT(*) > 5;
```

Explanation:
Filters grouped results, similar to `WHERE` but applied after grouping.
Purpose: To filter summarized/aggregated data rather than individual rows.
Real-time example: Management finds departments with more than 5 employees for restructuring.
Tester's POV: Testers use `HAVING` to verify threshold-based alerts or business rules, like flagging departments exceeding a headcount limit.

15. UPDATE

Syntax:

```sql
UPDATE table_name SET column_name = value WHERE condition;
```

Example:

```sql
UPDATE Employees SET Salary = 60000 WHERE EmployeeID = 1;
```

Explanation:
Modifies existing data in a table based on a condition.
Purpose: To keep records accurate and up to date as details change.
Real-time example: Payroll updates Anitha's salary after her annual appraisal.
Tester's POV: Testers directly update backend data to set up specific test preconditions (e.g., simulate an "expired" account) without going through the UI.

16. DELETE

Syntax:

```
DELETE FROM table_name WHERE condition;
```

Example:

```
DELETE FROM Employees WHERE EmployeeID = 1;
```

Explanation:
Removes rows from a table that match a condition.
Purpose: To remove records that are no longer needed, like resigned employees.
Real-time example: HR deletes an employee's record after they resign from the company.
Tester's POV: Testers clean up test data after automation runs with `DELETE` to keep the test environment consistent for the next run.

17. ALTER TABLE

Syntax:

```
ALTER TABLE table_name ADD column_name datatype;
```

Example:

```
ALTER TABLE Employees ADD Email VARCHAR(100);
```

Explanation:
Changes the structure of an existing table, such as adding, modifying, or dropping columns.
Purpose: To evolve the table structure as new business requirements appear.
Real-time example: IT adds an `Email` column to store official addresses after email rollout.
Tester's POV: Testers verify that schema changes (like a new column) don't break existing automation scripts or API contracts.

18. DROP

Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Employees;
```

Explanation:
Permanently deletes an entire table (or database) along with its data.
Purpose: To completely remove obsolete tables or databases from the system.
Real-time example: A dev team drops an old `Employees_Backup2019` table no longer in use.
Tester's POV: Testers use `DROP` cautiously, usually only in isolated test/sandbox environments, to reset a schema completely before a fresh test cycle.

19. TRUNCATE

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE Employees;
```

Explanation:
Quickly removes all rows from a table while keeping its structure intact.
Purpose: To reset a table's data quickly without deleting its design.
Real-time example: A test team truncates the `Employees` table before loading fresh test data.
Tester's POV: Testers truncate tables between automated test suite runs to ensure each run starts from a clean, predictable state.

20. Aggregate Functions

Syntax:

```
SELECT COUNT(column_name), SUM(column_name), AVG(column_name), MIN(column_name),
MAX(column_name)
FROM table_name;
```

Example:

```
SELECT AVG(Salary) FROM Employees;
```

Explanation:
Performs a calculation (count, sum, average, min, max) across multiple rows and returns a single value.
Purpose: To get quick statistical insights from large datasets.
Real-time example: Finance calculates the average salary across the company for budgeting.
Tester's POV: Testers use aggregate functions to validate calculated fields shown in the UI, like total counts or sums on a dashboard.

21. JOINS

Syntax:

```
SELECT column_name FROM table_name1
JOIN table_name2 ON table_name1.column_name = table_name2.column_name;
```

Example:

```
SELECT Employees.Name, Departments.DeptName
FROM Employees
JOIN Departments ON Employees.DeptID = Departments.DeptID;
```

Explanation:
Combines rows from two or more tables based on a related column (types: `INNER`, `LEFT`, `RIGHT`, `FULL`).
Purpose: To combine related data spread across multiple tables into one result.
Real-time example: An HR dashboard shows each employee's name alongside their department name.
Tester's POV: Testers use joins to validate end-to-end data flow across related tables, ensuring foreign key relationships display correctly in the app.

22. Subqueries

Syntax:

```
SELECT column_name FROM table_name WHERE column_name = (SELECT column_name FROM
table_name WHERE condition);
```

Example:

```
SELECT Name FROM Employees WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Explanation:
A query nested inside another query, used to filter or compute values dynamically.
Purpose: To use the result of one query as input for another, more complex query.
Real-time example: Management lists employees earning above the company's average salary.
Tester's POV: Testers write subqueries to build dynamic validation checks instead of hardcoding expected values in test scripts.

23. CASE

Syntax:

```sql
SELECT column_name,
CASE
    WHEN condition THEN result
    ELSE result
END AS alias_name
FROM table_name;
```

Example:

```sql
SELECT Name,
CASE
    WHEN Salary > 50000 THEN 'High'
    ELSE 'Low'
END AS SalaryLevel
FROM Employees;
```

Explanation:
Adds conditional (if-else style) logic directly inside a SQL query.
Purpose: To classify or label data dynamically based on conditions.
Real-time example: A report tags each employee's salary as "High" or "Low" for quick review.
Tester's POV: Testers use `CASE` to build expected-vs-actual comparison logic directly in validation queries.

24. UNION

Syntax:

```sql
SELECT column_name FROM table_name1
UNION
SELECT column_name FROM table_name2;
```

Example:

```sql
SELECT Name FROM Employees
UNION
SELECT Name FROM Contractors;
```

Explanation:
Combines results of two `SELECT` queries into one, removing duplicate rows (`UNION ALL` keeps duplicates).
Purpose: To merge similar data from different tables into a single unified list.
Real-time example: HR creates a combined list of full-time employees and contractors for an event.
Tester's POV: Testers use `UNION` to compare data sets from two environments (e.g., staging vs production) in one combined query.

25. EXISTS

Syntax:

```
SELECT column_name FROM table_name WHERE EXISTS (SELECT column_name FROM table_name
WHERE condition);
```

Example:

```
SELECT Name FROM Employees WHERE EXISTS
(SELECT 1 FROM Departments WHERE Departments.DeptID = Employees.DeptID);
```

Explanation:
Checks whether a subquery returns any rows, returning `TRUE` or `FALSE`.
Purpose: To efficiently check for the presence of related data before including a row.
Real-time example: The system lists only employees who belong to a department that still exists.
Tester's POV: Testers use `EXISTS` to quickly assert whether a specific test record was created/deleted successfully, without pulling full data.

26. CTE (Common Table Expression)

Syntax:

```
WITH cte_name AS (
    SELECT column_name FROM table_name WHERE condition
)
SELECT * FROM cte_name;
```

Example:

```
WITH HighEarners AS (
    SELECT Name, Salary FROM Employees WHERE Salary > 50000
)
SELECT * FROM HighEarners;
```

Explanation:
Creates a temporary, named result set that can be referenced within a query, making complex queries easier to read.
Purpose: To simplify and organize complex, multi-step queries into readable blocks.
Real-time example: A report builder first isolates high earners with a CTE, then analyzes them further.
Tester's POV: Testers use CTEs to break down complex data validation logic into readable steps, making automated DB assertions easier to debug.

27. Views

Syntax:

```
CREATE VIEW view_name AS
SELECT column_name FROM table_name WHERE condition;
```

Example:

```sql
CREATE VIEW ITEmployees AS
SELECT Name, Salary FROM Employees WHERE Department = 'IT';
```

Explanation:
Saves a query as a virtual table that can be reused like a regular table without storing actual data.
Purpose: To simplify repeated access to a specific, filtered dataset.
Real-time example: IT managers query the `ITEmployees` view daily instead of rewriting the filter each time.
Tester's POV: Testers create views to expose a stable, simplified dataset for automated test assertions, hiding underlying schema complexity.

28. Indexes

Syntax:

```sql
CREATE INDEX index_name ON table_name (column_name);
```

Example:

```sql
CREATE INDEX idx_name ON Employees (Name);
```

Explanation:
Speeds up data retrieval on a column at the cost of extra storage and slower writes.
Purpose: To make searches and lookups on large tables much faster.
Real-time example: An HR portal indexes the `Name` column so employee search loads instantly.
Tester's POV: Testers verify performance test results before/after indexing to confirm query response times meet non-functional requirements.

29. Primary Key

Syntax:

```sql
CREATE TABLE table_name (
    column_name datatype PRIMARY KEY
);
```

Example:

```sql
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50)
);
```

Explanation:
Uniquely identifies each row in a table; cannot be `NULL` or duplicated.
Purpose: To guarantee every record can be uniquely and reliably identified.

Real-time example: Each employee is uniquely tracked across systems using their `EmployeeID`.
Tester's POV: Testers write negative test cases to confirm duplicate or NULL primary key inserts are correctly rejected by the system.

30. Foreign Key

Syntax:

```
CREATE TABLE table_name (
    column_name datatype,
    FOREIGN KEY (column_name) REFERENCES other_table(column_name)
);
```

Example:

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

Explanation:
Links a column to the primary key of another table, enforcing referential integrity between tables.
Purpose: To maintain valid, consistent relationships between related tables.
Real-time example: An employee's `DeptID` must match a real department, preventing invalid department entries.
Tester's POV: Testers try inserting invalid foreign key values to confirm the system properly blocks orphaned or broken references.

31. Constraints

Syntax:

```
CREATE TABLE table_name (
    column_name datatype NOT NULL UNIQUE CHECK (condition) DEFAULT value
);
```

Example:

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Salary DECIMAL(10,2) CHECK (Salary > 0) DEFAULT 30000
);
```

Explanation:
Rules applied to columns to enforce data validity, such as `NOT NULL`, `UNIQUE`, `CHECK`, and `DEFAULT`.
Purpose: To protect data quality by preventing invalid or incomplete entries.
Real-time example: The system blocks any employee record from being saved with a negative

salary.
Tester's POV: Testers specifically design negative tests around constraints, like trying to insert a negative salary, to confirm validation rules work.

32. Transactions

Syntax:

```
BEGIN TRANSACTION;
    -- SQL statements
COMMIT;
-- or
ROLLBACK;
```

Example:

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = Salary + 5000 WHERE Department = 'IT';
COMMIT;
```

Explanation:
Groups multiple SQL statements so they all succeed together (`COMMIT`) or all fail together (`ROLLBACK`), ensuring data consistency.
Purpose: To ensure a group of related changes either fully applies or doesn't apply at all.
Real-time example: A payroll run rolls back the entire salary hike if any single update fails midway.
Tester's POV: Testers wrap test data setup/teardown in transactions with `ROLLBACK` so automated tests never leave permanent side effects in the database.

Part 2: Advanced Topics

33. Window Functions

Syntax:

```
SELECT column_name,
FUNCTION() OVER (PARTITION BY column_name ORDER BY column_name)
FROM table_name;
```

Example:

```
SELECT Name, Department, Salary,
RANK() OVER (PARTITION BY Department ORDER BY Salary DESC) AS SalaryRank
FROM Employees;
```

Explanation:
Performs a calculation across a set of related rows without collapsing them into a single result, unlike `GROUP BY`.
Purpose: To rank, compare, or run running totals on rows while still showing every individual row.
Real-time example: HR ranks employees by salary within each department to find the top earner per team.
Tester's POV: Testers use window functions to validate ranking/leaderboard features by comparing app output against the database-calculated rank.

34. ROW_NUMBER() / LEAD() / LAG()

Syntax:

```
SELECT column_name,
ROW_NUMBER() OVER (ORDER BY column_name) AS row_num,
LAG(column_name) OVER (ORDER BY column_name) AS prev_value,
LEAD(column_name) OVER (ORDER BY column_name) AS next_value
FROM table_name;
```

Example:

```
SELECT Name, Salary,
ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum
FROM Employees;
```

Explanation:
`ROW_NUMBER()` assigns a unique sequential number to each row; `LAG()`/`LEAD()` fetch values from previous/next rows.
Purpose: To number results, remove duplicates, or compare a row's value with the row before/after it.
Real-time example: A report lists employees numbered by salary rank to identify the top 3

earners.
Tester's POV: Testers use `ROW_NUMBER()` to easily fetch and validate the "latest" or "nth" test record, e.g., checking the most recent audit log entry.

35. String Functions

Syntax:

```
SELECT CONCAT(column_name, column_name), UPPER(column_name), LOWER(column_name),
SUBSTRING(column_name, start, length), TRIM(column_name)
FROM table_name;
```

Example:

```
SELECT CONCAT(Name, ' - ', Department) AS EmployeeInfo FROM Employees;
```

Explanation:
Functions that manipulate text values, such as combining, changing case, extracting parts, or trimming spaces.
Purpose: To clean, format, or combine text data for reports and display.
Real-time example: A dashboard shows each employee's name and department combined in a single label.
Tester's POV: Testers use string functions to detect data issues like extra whitespace, wrong casing, or bad formatting saved by the application.

36. Date Functions

Syntax:

```
SELECT column_name, DATEDIFF(date1, date2), NOW(), YEAR(column_name),
MONTH(column_name)
FROM table_name;
```

Example:

```
SELECT Name, DATEDIFF(CURDATE(), JoiningDate) AS DaysEmployed FROM Employees;
```

Explanation:
Functions that calculate, extract, or format date and time values.
Purpose: To calculate durations, ages, or extract parts of a date for reporting.
Real-time example: HR calculates how many days each employee has worked since joining.
Tester's POV: Testers use date functions to validate time-based automation logic, like credential rotation intervals or session expiry timestamps.

37. Math / Numeric Functions

Syntax:

```
SELECT ROUND(column_name, decimals), ABS(column_name), CEILING(column_name),
FLOOR(column_name)
FROM table_name;
```

Example:

```
SELECT Name, ROUND(Salary, 0) AS RoundedSalary FROM Employees;
```

Explanation:
Functions that perform mathematical operations like rounding, absolute value, and ceiling/floor calculations.
Purpose: To format or adjust numeric values for calculations and clean display.
Real-time example: Payroll rounds off salary figures to whole numbers for a printed payslip.
Tester's POV: Testers use these functions to verify rounding/precision bugs in financial calculations shown by the application.

38. NULL Handling Functions

Syntax:

```
SELECT COALESCE(column_name, default_value), NULLIF(column_name, value)
FROM table_name;
```

Example:

```
SELECT Name, COALESCE(ManagerID, 0) AS ManagerID FROM Employees;
```

Explanation:
`COALESCE` returns the first non-null value from a list; `NULLIF` returns `NULL` if two values are equal.
Purpose: To provide fallback values and avoid `NULL` causing errors in calculations or reports.
Real-time example: A report shows "0" instead of blank for employees who have no manager assigned.
Tester's POV: Testers use `COALESCE` in validation queries to safely compare fields that may sometimes be `NULL`, avoiding false test failures.

39. Data Types (Overview)

Syntax:

```
column_name INT | BIGINT | DECIMAL(p,s) | VARCHAR(n) | TEXT | DATE | DATETIME |
BOOLEAN
```

Example:

```sql
CREATE TABLE Employees (
    EmployeeID INT,
    Name VARCHAR(50),
    JoiningDate DATE,
    Salary DECIMAL(10,2)
);
```

Explanation:
Data types define what kind of value a column can store — numbers, text, dates, or true/false values.
Purpose: To ensure each column stores correct, consistent, and space-efficient data.
Real-time example: The `Employees` table uses `DATE` for `JoiningDate` so only valid calendar dates are allowed.
Tester's POV: Testers write boundary tests around data types, like inserting oversized text or invalid dates, to confirm the system rejects bad input.

40. Stored Procedures

Syntax:

```sql
CREATE PROCEDURE procedure_name (parameter datatype)
AS
BEGIN
    SQL statements;
END;
```

Example:

```sql
CREATE PROCEDURE GetEmployeesByDept (@Dept VARCHAR(50))
AS
BEGIN
    SELECT Name FROM Employees WHERE Department = @Dept;
END;
```

Explanation:
A saved, reusable block of SQL code that can be executed with a single call, optionally accepting parameters.
Purpose: To reuse common logic and reduce repeated queries from the application side.
Real-time example: An app calls `GetEmployeesByDept('IT')` instead of writing the full query every time.
Tester's POV: Testers call stored procedures directly to quickly set up or validate complex test scenarios without duplicating the logic in test scripts.

41. User-Defined Functions

Syntax:

```
CREATE FUNCTION function_name (parameter datatype)
RETURNS datatype
AS
BEGIN
    RETURN value;
END;
```

Example:

```
CREATE FUNCTION GetAnnualSalary (@Monthly DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Monthly * 12;
END;
```

Explanation:
A reusable block of logic that returns a single value or table, callable within SQL statements like a built-in function.
Purpose: To encapsulate repeated calculations so they can be reused across many queries.
Real-time example: Payroll uses `GetAnnualSalary(Salary)` to instantly compute yearly pay for every employee.
Tester's POV: Testers reuse user-defined functions in validation queries to keep expected-value calculations consistent with production logic.

42. Triggers

Syntax:

```
CREATE TRIGGER trigger_name
AFTER INSERT|UPDATE|DELETE ON table_name
FOR EACH ROW
BEGIN
    SQL statements;
END;
```

Example:

```
CREATE TRIGGER LogSalaryChange
AFTER UPDATE ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO SalaryAudit (EmployeeID, OldSalary) VALUES (OLD.EmployeeID,
OLD.Salary);
END;
```

Explanation:
A block of code that automatically runs in response to a specific event (insert, update, delete) on a table.
Purpose: To automatically enforce rules or log changes without relying on the application code.

Real-time example: Every time an employee's salary is updated, the old salary is auto-logged for audit purposes.
Tester's POV: Testers verify triggers fire correctly by performing an action and then checking the audit/log table for the expected auto-generated entry.

43. Self Join

Syntax:

```
SELECT a.column_name, b.column_name
FROM table_name a, table_name b
WHERE a.column_name = b.column_name;
```

Example:

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Explanation:
Joins a table to itself, treating it as two separate tables using aliases.
Purpose: To compare rows within the same table, such as hierarchical relationships.
Real-time example: An org chart report shows each employee alongside the name of their manager.
Tester's POV: Testers use self joins to validate hierarchical UI features, like manager-employee mapping shown in an org chart screen.

44. Recursive CTE

Syntax:

```
WITH RECURSIVE cte_name AS (
    SELECT column_name FROM table_name WHERE condition
    UNION ALL
    SELECT column_name FROM table_name JOIN cte_name ON condition
)
SELECT * FROM cte_name;
```

Example:

```
WITH RECURSIVE OrgChart AS (
    SELECT EmployeeID, Name, ManagerID FROM Employees WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmployeeID, e.Name, e.ManagerID
    FROM Employees e
    JOIN OrgChart o ON e.ManagerID = o.EmployeeID
)
SELECT * FROM OrgChart;
```

Explanation:
A CTE that references itself repeatedly to work through hierarchical or tree-structured data.
Purpose: To traverse multi-level hierarchies like org charts or category trees in one query.
Real-time example: HR generates a full reporting chain from the CEO down to every employee.
Tester's POV: Testers use recursive CTEs to validate deeply nested data (like multi-level approval workflows) end-to-end in one query.

45. GRANT / REVOKE (Permissions)

Syntax:

```
GRANT permission ON table_name TO user_name;
REVOKE permission ON table_name FROM user_name;
```

Example:

```
GRANT SELECT, UPDATE ON Employees TO hr_user;
```

Explanation:
`GRANT` gives a user specific access rights to a database object; `REVOKE` removes those rights.
Purpose: To control who can view or modify sensitive data.
Real-time example: Only the `hr_user` account is allowed to view and update the `Employees` table.
Tester's POV: Testers verify role-based access control by confirming a restricted user account cannot perform actions beyond their granted permissions.

46. Normalization (Design Concept)

Syntax:

```
-- Not a command; a design principle applied while creating tables
-- 1NF: atomic columns | 2NF: no partial dependency | 3NF: no transitive dependency
```

Example:

```
-- Instead of one table with repeating Department info,
-- split into Employees(DeptID) and Departments(DeptID, DeptName)
```

Explanation:
A design process that organizes tables to reduce data redundancy and improve data integrity.
Purpose: To avoid duplicate data and keep the database easy to maintain and update.
Real-time example: Department names are stored once in a `Departments` table instead of repeating in every employee row.
Tester's POV: Testers review normalized schema design to plan realistic multi-table join validations instead of assuming flat, single-table data.

Quick Reference Summary

Category	Commands
Database/Table Setup	CREATE DATABASE, CREATE TABLE, ALTER TABLE, DROP, TRUNCATE
Data Manipulation	INSERT, UPDATE, DELETE
Data Retrieval	SELECT, DISTINCT, WHERE, ORDER BY, GROUP BY, HAVING
Filtering	AND/OR/NOT, BETWEEN, IN, LIKE, IS NULL
Calculations	Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
Combining Data	JOINS, UNION, Subqueries, EXISTS, CTE, Self Join, Recursive CTE
Logic	CASE
Performance/Structure	Indexes, Views, Primary Key, Foreign Key, Constraints
Reliability	Transactions
Analytics	Window Functions, ROW_NUMBER/LEAD/LAG
Data Formatting	String Functions, Date Functions, Math Functions, NULL Handling
Design	Data Types, Normalization
Reusable Logic	Stored Procedures, User-Defined Functions, Triggers
Security	GRANT / REVOKE

SQL Interview Quick Reference

A fast-revision cheat sheet of commonly asked interview syntax and concepts.

Note on syntax style — Single-Table vs Multi-Table:

You'll notice the `Syntax` column below writes tables two different ways, and this is intentional, not inconsistent:

- **Single-Table syntax** uses the placeholder `table_name` directly (e.g., `FROM table_name`) because only one table is involved — there's no ambiguity, so the full name is used as-is.
- **Multi-Table syntax** (joins, unions, self joins, correlated subqueries) uses short aliases like `A`, `B`, or `table_name a` / `table_name b` (e.g., `FROM A JOIN B ON A.id = B.id`) because two or more tables — sometimes the *same* table twice — are referenced in one query. Aliases keep column references (`A.id` vs `B.id`) short and unambiguous; writing the full `table_name.column_name` every time would make joins hard to read. This is also how aliases are actually written in real production SQL and in interviews.

The **Scope** column below tells you which style to expect at a glance.

Topic	Scope	Syntax	One-line Answer
Basic SELECT	**Single-Table**	`SELECT column_name FROM table_name WHERE condition;`	Retrieves filtered data from a table.
SELECT TOP / LIMIT	**Single-Table**	`SELECT * FROM table_name LIMIT n;`	Restricts the number of rows returned.
INNER JOIN	**Multi-Table**	`SELECT * FROM A INNER JOIN B ON A.id = B.id;`	Returns only matching rows from both tables.
LEFT JOIN	**Multi-Table**	`SELECT * FROM A LEFT JOIN B ON A.id = B.id;`	Returns all rows from left table + matched rows from right.
RIGHT JOIN	**Multi-Table**	`SELECT * FROM A RIGHT JOIN B ON A.id = B.id;`	Returns all rows from right table + matched rows from left.
FULL JOIN	**Multi-Table**	`SELECT * FROM A FULL JOIN B ON A.id = B.id;`	Returns all rows when there's a match in either table.
Self Join	**Multi-Table (same table twice)**	`SELECT a.col, b.col FROM table_name a JOIN table_name b ON a.ref = b.id;`	Joins a table with itself for hierarchical data.
WHERE vs HAVING	**Single-Table**	`... WHERE condition GROUP BY col HAVING agg_condition;`	`WHERE` filters rows; `HAVING` filters grouped/aggregated results.
GROUP BY	**Single-Table**	`SELECT col, COUNT(*) FROM table_name GROUP BY col;`	Groups rows to apply aggregate functions per group.
DISTINCT vs GROUP BY	**Single-Table**	`SELECT DISTINCT col FROM table_name;`	`DISTINCT` removes duplicates; `GROUP BY` groups for aggregation.
Primary Key vs Foreign Key	**Multi-Table (reference)**	`PRIMARY KEY(col)` / `FOREIGN KEY(col) REFERENCES other_table(col)`	Primary key uniquely identifies a row; foreign key links to another table.
DELETE vs TRUNCATE vs DROP	**Single-Table**	`DELETE FROM table_name WHERE condition;` / `TRUNCATE TABLE table_name;` / `DROP TABLE table_name;`	DELETE removes rows (logged, filterable); TRUNCATE clears all rows fast; DROP removes the whole table.
UNION vs UNION ALL	**Multi-Table**	`SELECT col FROM A UNION SELECT col FROM B;`	`UNION` removes duplicates; `UNION ALL` keeps them (faster).
Subquery	**Single-Table (nested)**	`SELECT col FROM table_name WHERE col = (SELECT ...);`	A query nested inside another query.

Correlated Subquery	**Multi-Table**	`SELECT col FROM A WHERE EXISTS (SELECT 1 FROM B WHERE B.id = A.id);`	A subquery that references the outer query's row.
CTE	**Single-Table (named block)**	`WITH cte_name AS (SELECT ...) SELECT * FROM cte_name;`	A named temporary result set for readable, reusable queries.
Window Function	**Single-Table**	`RANK() OVER (PARTITION BY col ORDER BY col);`	Calculates across related rows without collapsing them.
Index	**Single-Table**	`CREATE INDEX index_name ON table_name(column_name);`	Speeds up lookups on a column at the cost of write performance.
View	**Single-Table (virtual)**	`CREATE VIEW view_name AS SELECT ... ;`	A saved virtual table based on a query.
Transaction / ACID	**Single-Table**	`BEGIN TRANSACTION; ... COMMIT; / ROLLBACK;`	Ensures Atomicity, Consistency, Isolation, Durability of DB operations.
Normalization	**Multi-Table (design)**	`1NF → 2NF → 3NF`	Organizes tables to reduce redundancy and improve integrity.
Constraints	**Single-Table**	`NOT NULL, UNIQUE, CHECK(condition), DEFAULT value`	Rules that enforce valid data at the column level.
Stored Procedure	**Single-Table**	`CREATE PROCEDURE procedure_name (@param) AS BEGIN ... END;`	A reusable, saved block of SQL logic.
Trigger	**Single-Table**	`CREATE TRIGGER trigger_name AFTER INSERT ON table_name ...`	Auto-executes logic in response to a table event.
Aggregate Functions	**Single-Table**	`COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`	Perform calculations across multiple rows into one value.
NULL Handling	**Single-Table**	`COALESCE(col, default_value)`	Returns a fallback value when the column is `NULL`.
Clause Execution Order	**Single-Table**	`FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`	The logical order SQL processes a query internally.

> **Tip:** A very common interview question is "*What is the difference between WHERE and HAVING?*" or "*Explain the order of execution of a SQL query.*" — both are covered above for quick revision before an interview.

Interview Quick Reference — Tanglish Explanation

Same 25 topics, explained in simple Tanglish (Tamil + English) for quick, easy revision.

Topic	Tanglish Explanation
Basic SELECT	Condition match aagira rows-a table-la irundhu edukurathukku `SELECT` use pannuvom.
SELECT TOP / LIMIT	Evlo rows venumo, adha mattum kaata `LIMIT` use pannuvom.
INNER JOIN	Rendu table-layum match aagira rows mattum thaan result-la varum.
LEFT JOIN	Left table full-a varum, right table-la match aana rows mattum add aagum.
RIGHT JOIN	Right table full-a varum, left table-la match aana rows mattum add aagum.
FULL JOIN	Rendu table-layum match aana rows-um, match aagaadha rows-um serthu varum.
Self Join	Same table-a rendu different alias vachi join pannuvom — manager-employee madhiri hierarchy data-ku use aagum.
WHERE vs HAVING	`WHERE` row level-la filter pannum; `HAVING` group pannina appuram (aggregate) filter pannum.
GROUP BY	Same value irukra rows-a oru group-a vachi COUNT, SUM madhiri calculation pannuvom.
DISTINCT vs GROUP BY	`DISTINCT` duplicate values-a remove pannum; `GROUP BY` calculation panna rows-a group pannum.
Primary Key vs Foreign Key	Primary key oru row-a unique-a identify pannum; foreign key innoru table-oda relation create pannum.
DELETE vs TRUNCATE vs DROP	`DELETE` condition vachi rows mattum neekum; `TRUNCATE` ella rows-um fast-a clear pannum; `DROP` full table-ye remove pannum.
UNION vs UNION ALL	`UNION` duplicate rows-a remove pannum; `UNION ALL` duplicate-um vachikkum, adhanala fast-a work aagum.
Subquery	Oru query-kulla innoru query irukkum, adha result-a use pannuvom.
Correlated Subquery	Outer query-oda ovvoru row-a base pannii, inner query repeat-a run aagum.
CTE	Complex query-a padikka easy-a irukka, temporary named result vachi `WITH` use pannuvom.
Window Function	Rows-a collapse pannaama, group calculation pannalam — rank, running total madhiri.
Index	Column-la search fast-a nadakka structure create pannuvom, aana write speed konjam slow aagum.
View	Query-a save pannii, adha virtual table madhiri repeat-a use pannuvom.
Transaction / ACID	Steps ellam onna succeed aaganum, illana ellam fail aaganum — data safe-a irukka idhu help pannum.
Normalization	Duplicate data kammiya-panna table structure-a proper-a organize pannuradhu.
Constraints	Column-la correct data mattum varanum-nu rules vekkalam — `NOT NULL`, `UNIQUE`, `CHECK` madhiri.
Stored Procedure	Repeat-a use aagira SQL logic-a save pannii, easy-a call panna mudiyum.
Trigger	Insert/update/delete event nadandha udane automatic-a run aagira code.
Aggregate Functions	Pala rows-a serthu oru single value-a calculate pannuradhu — `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
NULL Handling	Value illama irukra edathula fallback value kuduka `COALESCE` use pannuvom.
Clause Execution Order	SQL query-a andharukulla eppadi process pannum-nu order — `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`.

> **Tip:** Interview-la konjam relax-a irukkanum-na, idhu madhiri Tanglish-la first puriyavachukittu, apparam English technical term-a nyabagam vechukonga.